

OE-EC604C — Object Oriented Programming

Semester VI • 3L:0T:0P • 3 Credits

Module 9 Exam Notes: Exception Handling & Templates

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Exception Handling Mechanism	2
1.1	Basic Syntax: <code>try</code> , <code>throw</code> , and <code>catch</code>	2
1.2	Exception Propagation & Stack Unwinding	2
1.3	Multiple Catches, Catch-All, and Re-throwing	3
2	Generic Programming using Templates	3
2.1	Function Templates	3
2.2	Class Templates	4

1. Exception Handling Mechanism

Exception handling separates error detection from error resolution, allowing applications to recover gracefully from runtime errors.

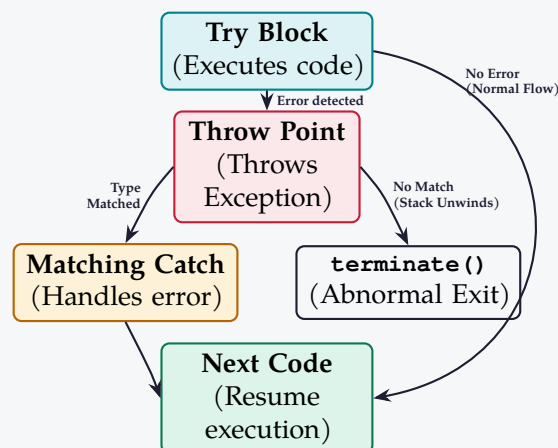
1.1 Basic Syntax: `try`, `throw`, and `catch`

- **`try` block:** Encloses the segment of code where exceptions might occur.
- **`throw` statement:** Triggers an exception when an anomaly is detected, exiting the `try` block immediately.
- **`catch` block:** Defines a handler that matches the data type of the thrown exception to resolve the error.

1.2 Exception Propagation & Stack Unwinding

If a `throw` point occurs inside a nested function, the compiler exits that function immediately. It searches up the function call stack for an active `try` block. During this process, the compiler invokes destructors for all local variables created inside the exited stack frames. This process is called **Stack Unwinding**.

Exception Handling Control Flow Map



Code Example: Exception throw point outside the main try block

```

#include <iostream>
// Function that throws exception
double divide(double a, double b) {
    if(b == 0.0) {
        throw "Division by zero condition!"; // Throws a const char*
    }
    return a / b;
}

int main() {
    double x = 10, y = 0, result;
    try {

```

```

        result = divide(x, y); // Throw point is inside divide()
        std::cout << "Result = " << result << std::endl;
    }
    catch(const char* msg) { // Catches the thrown char*
        std::cerr << "Caught Exception: " << msg << std::endl;
    }
    return 0;
}

```

1.3 Multiple Catches, Catch-All, and Re-throwing

- **Multiple Catches:** A single try block can have multiple sequential catch handlers to process different exception types.
- **Catch-All (`catch(...)`):** A special catch handler that intercepts any exception type. It must be placed at the very end of the handler list.
- **Exception Re-throwing:** An caught exception can be re-thrown using the `throw;` statement without arguments inside the catch block to pass it up to a higher scope.

2. Generic Programming using Templates

Templates allow writing code that is independent of data types, enabling the creation of generic functions and classes.

2.1 Function Templates

Function templates act as formulas for generating functions based on argument types.

Function Template Syntax

```
template <typename T>    Return_Type  FuncName(T args)  {...}    (1)
```

Code Example: Multi-parameter function template

```

#include <iostream>
// Multi-parameter function template
template <typename T1, typename T2>
void printComparison(T1 x, T2 y) {
    if(x > y) {
        std::cout << x << " is larger than " << y << std::endl;
    } else {
        std::cout << x << " is not larger than " << y << std::endl;
    }
}

int main() {

```

```
printComparison(10, 5.5); // T1 = int, T2 = double
printComparison(4.2, 9.8); // T1 = double, T2 = double
return 0;
}
```

2.2 Class Templates

Class templates are used to create generic containers or data structures that operate on any data type.

Code Example: Generic Stack class template

```
#include <iostream>
template <typename T, int SIZE = 10> // typename T and non-type int SIZE
class Stack {
private:
    T arr[SIZE];
    int top;
public:
    Stack() : top(-1) {}

    bool push(T element) {
        if(top >= SIZE - 1) return false; // Overflow
        arr[++top] = element;
        return true;
    }

    T pop() {
        if(top < 0) throw "Stack Underflow!"; // Throw const char*
        return arr[top--];
    }
};

int main() {
    try {
        Stack<int, 5> intStack; // Instantiate an int stack of size 5
        intStack.push(10);
        intStack.push(20);
        std::cout << intStack.pop() << std::endl; // 20
        std::cout << intStack.pop() << std::endl; // 10
        std::cout << intStack.pop() << std::endl; // Throws exception
    }
    catch(const char* e) {
        std::cout << "Exception: " << e << std::endl;
    }
    return 0;
}
```

Quick Revision — Module IX

Exception Handling & Templates | OE-EC604C

Key Exception and Template Syntax

Try-Catch Block	<pre>try { throw 404; } catch(int err) { /* handles 404 */ }</pre>
Catch-All Handler	<pre>catch(...) { /* catches any exception type */ }</pre>
Re-throwing	<pre>catch(const char* e) { throw; // Re-throws exception up }</pre>
Function Template	<pre>template <typename T> T maxVal(T a, T b) { return (a > b) ? a : b; }</pre>
Class Template	<pre>template <class T> class Box { T value; };</pre>
Template Instantiation	<pre>Box<double> doubleBox; // Explicit template argument</pre>

One-Line Definitions

- **Exception:** A runtime error or anomaly that disrupts the normal execution flow of a program.
- **Stack Unwinding:** The automatic destruction of local variables in stack frames during exception propagation.
- **Catch-All Handler:** A fallback handler (`catch(...)`) that catches any exception type not previously intercepted.
- **Re-throwing:** Forwarding an caught exception up to the parent calling frame using the `throw;` statement.
- **Template:** A blueprint used by the compiler to generate type-specific functions or classes automatically.
- **Generic Programming:** A programming model focused on writing algorithms independent of specific data types.
- **Non-type Template Parameter:** A template parameter that represents a constant value (e.g., `int SIZE`) rather than a type.
- **Template Specialization:** Defining a unique template implementation for a specific data type (e.g., `T = char*`).
- **terminate():** A system function called when no matching catch handler is found for a thrown exception.

Frequently Asked / Viva Questions**Q1. What happens if an exception is thrown but no matching catch block is found?**

The runtime library calls the 'terminate()' function, which by default calls 'abort()' to terminate the program immediately.

Q2. Can we throw class objects as exceptions in C++?

Yes. Throwing class instances is the standard way to implement custom exceptions, allowing structured error data to be passed to handlers.

Q3. What is the purpose of the catch-all handler and where must it be placed?

'catch(...)' catches all exception types. It acts as a safety net and must be placed as the last handler, otherwise it blocks subsequent handlers.

Q4. Why does template instantiation not happen at class definition time?

A template only defines a blueprint. The compiler only generates class code when the template is instantiated with a specific type (e.g., 'Stack<int>').

Q5. Explain the compile-time overhead associated with templates.

Templates require the compiler to generate unique code for each type instantiation, which increases compile times and binary sizes (code bloat).

Q6. What is template function specialization?

It allows defining a custom implementation of a function template for a specific data type (e.g., optimizing string comparisons instead of pointer comparisons).

Q7. Can templates have default arguments?

Yes. Both class templates and function templates (in C++11) can have default type arguments (e.g., 'template <typename T = int>').

Q8. How does the compiler select between overloaded template functions and regular functions?

The compiler prefers a regular non-template function if it matches the arguments exactly. If no match is found, it instantiates the template.

Q9. Explain the difference between exception handling in C++ vs. Java.

C++ does not support checked exceptions (you do not have to declare exceptions in method signatures), and it does not have a native 'finally' block (resource cleanup relies on RAII).

Q10. What is the difference between 'typename' and 'class' keywords in template declarations?

Inside template parameters, 'typename' and 'class' are completely equivalent and interchangeable. E.g., 'template <class T>' and 'template <typename T>' behave identically.

Q11. Why is it dangerous to throw pointers to local variables?

Local variables are destroyed when stack frames are unwound. The handler will receive a dangling pointer to invalid memory, causing undefined behavior.

Frequently Asked / Viva Questions (contd.)

Q12. What is the RAII (Resource Acquisition Is Initialization) pattern?

It is a design pattern where resources (files, memory, locks) are bound to the lifetime of local objects. Destructors automatically release resources during normal exit or stack unwinding, preventing leaks.

Comparison: Exception Handling vs. Traditional Error Return Codes

Feature	Exception Handling (try-catch)	Traditional Error Codes (if-else)
Separation	Clear separation between execution logic and error handling code.	Error checking logic is mixed directly into the main algorithm.
Propagation	Errors propagate automatically up the call stack until handled.	Errors must be explicitly returned and checked in every function frame.
Data Payload	Exceptions can pass complex objects containing detailed error info.	Limited to returning simple integer codes or status booleans.
Performance	Slower when exceptions occur (due to stack unwinding and V-Table lookup).	Minimal overhead (simple conditional branches).
Uncaught Errors	Leads to immediate program termination (safe fail-fast).	Can be silently ignored, causing latent bugs or data corruption.